Syracuse University



Package organization and distribution

IST 400/600 Fall 2026

Announcements

- I will be away next week
- In lieu of class, we will have short (~15 min.) one-on-one meetings on Zoom
  - You will get your attendance point when you meet with me
- We will discuss progress on your project, any questions you have
- I will circulate a Google Doc on Blackboard to signup for a time
- Meetings will be between 10:30AM – 12:30PM on Wednesday and Thursday.
  - Let me know if you cannot make any of those times!

Outline

1. Source vs installed
2. Packaging and deployment
3. Language-specific practices
    a. Python
    b. R
    c. Web

Source code

- When you clone a repository, you end up with a directory of **source code**
- Source code consists of the scripts you and other programmers write
- Scripts are human readable
- They are generally *not* the thing the computer reads

Programming language

- A script is written in a programming language
  - "A programming language is a formal (computer) language designed to communicate instructions to a computer" [Wikipedia]
  - Examples: Python, R, C, C++, JavaScript, HTML
- Computers do not understand human language, they understand binary; ex:
  - (human) "2+2 = 4"
  - (computer) 0000010 "+ in binary" 0000010 = 00000100
- Programming languages allow humans to write instructions for a computer.

Scripted vs Compiled Languages

- A program you write is just a text file
- This human-readable text file must be translated into a computer-readable *machine code*
- How this is done depends on the language
- Two different types of programming languages:
  - **Scripting languages**
  - **Compiled languages**

Compiled languages

- Examples: C, C++, Fortran, BASIC
- In compiled languages a program must first be *compiled* into a binary file containing machine code the computer can run
- These languages require a program (a compiler) to translate the source code into a binary file containing the machine code
- Example compilers: gcc, clang
- The resulting binary file can be directly executed by the computer

Installation directory

- Generally, the binary files are stored to a different directory than the source code -> *installation directory*
- Installation directory will usually consist of a `bin` folder (short for “binary”) that stores executables
- There will also be a `lib` folder (short for “library”) that stores modules that can be imported into other programs; provide helper functions
- The installation directory is the folder package managers like conda write to. You will have a different installation directory for each environment.

Scripting languages

- Examples: Python, R, JavaScript, bash
- Scripted languages blur the line between script and program
- For scripting languages, a program is interpreted into machine code on the fly, at run time. No compiling necessary.
- The script is still not what the computer reads!
- These languages require a program (an interpreter) to do the translation. Example: Python scripts require a python interpreter (a binary file) to be installed on the computer.

Installation directory for scripted languages

- Scripted languages do not require an installed directory, since the scripts themselves are read at run-time
- Still, it's typically good practice to "install" the scripts to a directory separate from the source repository.
  - Keeps running code independent of development work
  - Also allows you to manage different environments
- For scripted languages, "installing" mostly means copying the scripts into the installation directory.

- File structure in installed directory for scripted languages typically the same as compiled languages
- Executables (the files you might run on the command line) stored in the bin directory
- Modules (the files that contain helper functions that are imported into other programs) stored in the lib directory

What about HTML?

- JavaScript, HTML & associated CSS are also scripting languages
- Browser is the interpreter
- Since these are used to build websites, there is no installation
- Scripts are provided by a web server, which your browser interprets and displays when you visit the site

Packaging and deployment

Packaging and deployment

- So far we've been discussing how to install your own code
- How do you share code with others? In other words, how do you ***deploy*** your code?
- Multiple methods:
  - Download source code from GitHub
  - Via a package repository
  - Via data sharing platforms

Downloading from source

- Method is what it sounds like: clone a repo, install it
  - Some package managers (e.g., pip) will let you install source code by providing a URL, without needing to clone the repo
- Simple to share: just stick on GitHub (or some other publicly available website)

Installing specific versions

- It is important to be able cite specific versions of code, especially when working in a research environment
- You can install and cite specific versions by using commit hashes; e.g.:

```
git clone <repo>
cd <repo>
git checkout 18cc40380
<install>
```

- Note that you only need to refer to enough characters in the hash so that it can be uniquely identified

Git tags

- Referring to commit hashes is ugly; hard to remember
- To remedy this Git allows you to create *tags*
  - https://git-scm.com/book/en/v2/Git-Basics-Tagging
- Tags are human-readable labels at specific points in code
- For example, running `git tag` in a clone of numpy yields:

```
$ git tag

pre-removal-numpybook
v0.2.2
v0.3.0
...
```

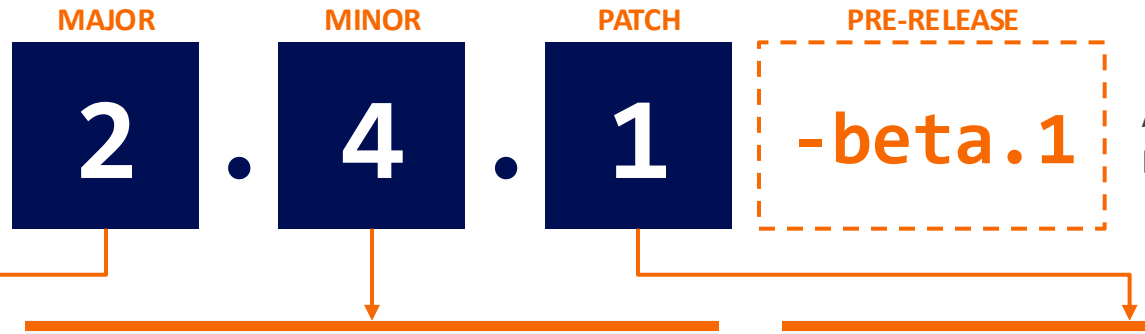
- You can then checkout / install a tag with `git checkout <tag>`

Software versioning

- Typically (but not required) tags will be version numbers
- Different schemes exist for numbering versions
  - https://en.wikipedia.org/wiki/Software_versioning
- Most common in open-source community is *semantic versioning* (**SemVer**)

SemVer

A SemVer number has three slots. Which slot changes tells users whether an upgrade is safe.



MAJOR

Breaking changes

Existing code may break: functions removed, renamed or changed.

`1.7.3 → 2.0.0`

Minor and patch reset to 0

MINOR

New features

Adds functionality; code written for the old version still works.

`2.3.5 → 2.4.0`

Patch resets to 0

PATCH

Bug fixes

Fixes incorrect behavior without changing the public API.

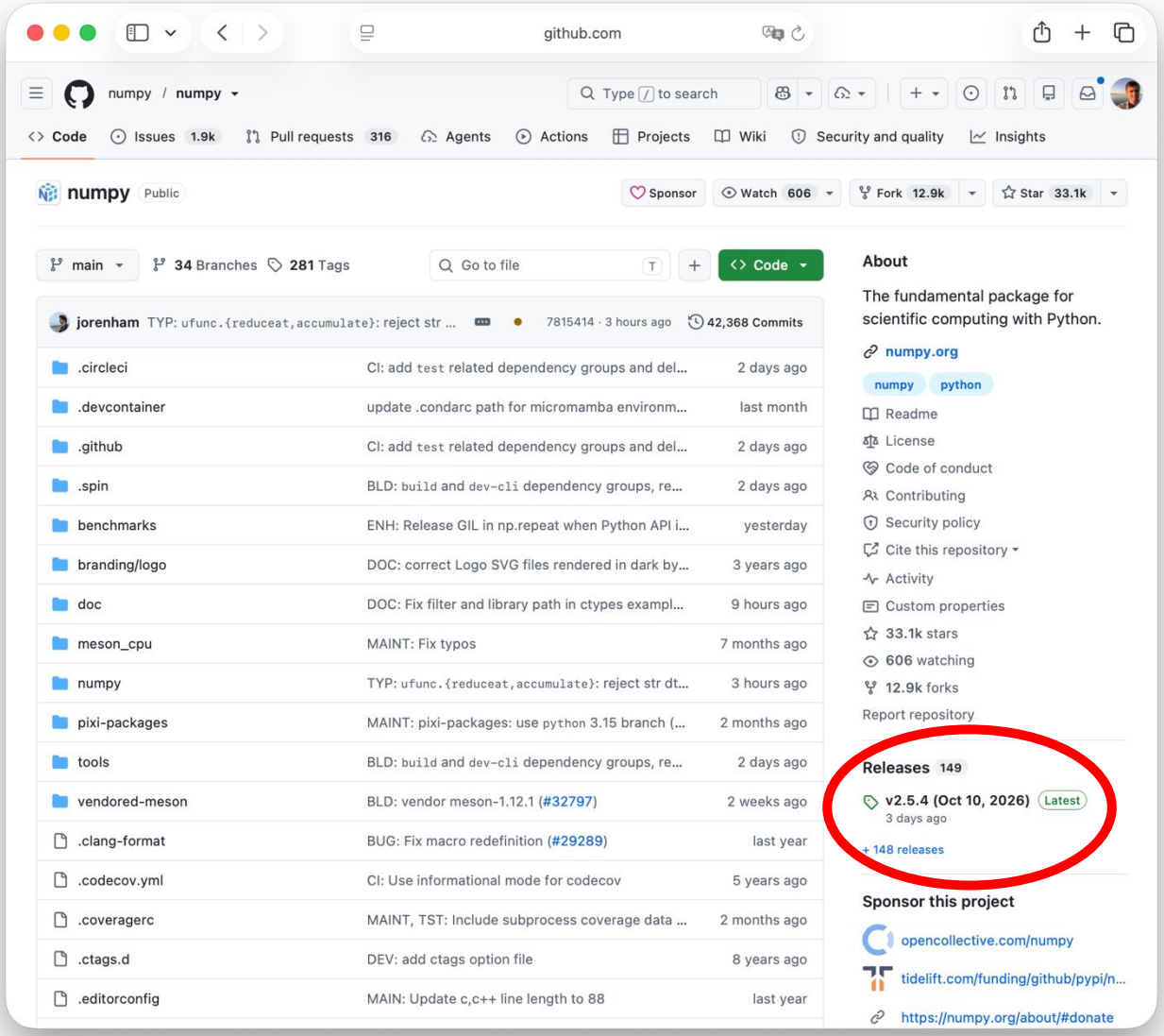
`2.4.0 → 2.4.1`

Safe to take right away

Versions below 1.0.0 promise no stability. Source: semver.org (SemVer 2.0.0)

GitHub release

- GitHub builds on tags, offering **releases**
  - https://docs.github.com/en/repositories/releasing-projects-on-github/about-releases
- These are tags with additional features
- Releases are linked from the repo's webpage



GitHub release

- A downloadable zip file of the package is created for releases
- Users can download and install the released zip file instead of needing to clone the entire repo
- A web page in which you can put release notes also created
- Example (from numpy): https://github.com/numpy/numpy/releases/tag/v2.5.4
- For instructions on how to create a release: https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository

Drawbacks to downloading from source

- Requires some technical know how
  - You're probably not going to get wide adoption this way
- You also need to know URLs for every package that you want to install
  - This will get ugly if you have a large number of dependencies
- Compiling code can be a pain; need right version of compiler
  - Less of an issue for scripted languages, but even these may have some internal compiled bits

Deploying via package repositories

- Better way to deploy is to upload to a package repository
- Language specific repositories exist (e.g. Python has PyPi)
- Method for uploading is different for each repo (more on that later)
- Once uploaded, can install with a simple command; e.g., to install numpy via PyPi:

```
pip install numpy
```

Deploying packages via conda-forge

- Can also upload to a conda repository
- **conda-forge** (https://conda-forge.org/) is the biggest in the open-source community, but other field-specific ones exist
- conda-forge is language agnostic: *any* package can be uploaded
  - conda-forge requires that you have an open source license though
- Packages are pre-compiled
- Uploading is more complicated as a result. For instructions, see: https://conda-forge.org/docs/maintainer/adding_pkgs/

DOIs

- Another method for deploying and versioning codes is to obtain a digital object identifier (DOI) for the package
  - https://www.doi.org/
- DOIs are unique numbers that double as URLs
- Most commonly used to uniquely identify research papers
- Ex: https://doi.org/10.48550/arXiv.1706.03762 points to the "Attention is all you need" paper on arXiv
- Most journals provide a DOI upon article publication

DOIs for software

- Services exist that will provide DOIs for software packages
- Example: Zenodo
  - https://help.zenodo.org/docs/github/enable-repository/
- Zenodo is used for data releases, but you can also upload software to it
- Can link GitHub to Zenodo, so that whenever a new release is created, the zip file is automatically uploaded to Zenodo, and a new DOI is assigned

Why DOIs?

- Obtaining DOIs for software are more common in academia
- Provide a way to cite software in publications
- This is important for reproducibility
- Also a way to get credit for doing software development
  - Publications are the currency of academia

Other ways to deploy code

- Docker containers
  - Highly reproducible, but more tricky to set up
- Post the code (possibly pre-compiled) on your website
  - This was more popular before GitHub; less common now
- Via proprietary app stores
  - Less common in the open-source community & academic research

An example workflow

- Have a file on main branch that sets the version of the code to X.X.Xdev (e.g., pyproject.toml in Python)
- Develop code through PRs
- When ready for a release (via roadmap, a requested feature, or a bug fix), drop the “dev” from the version number, then create a release on GitHub with the same number.
- Deploy release to package repositories
- Update version file to X.{X+1}.0dev on main; repeat.

Language-specific standards

Python

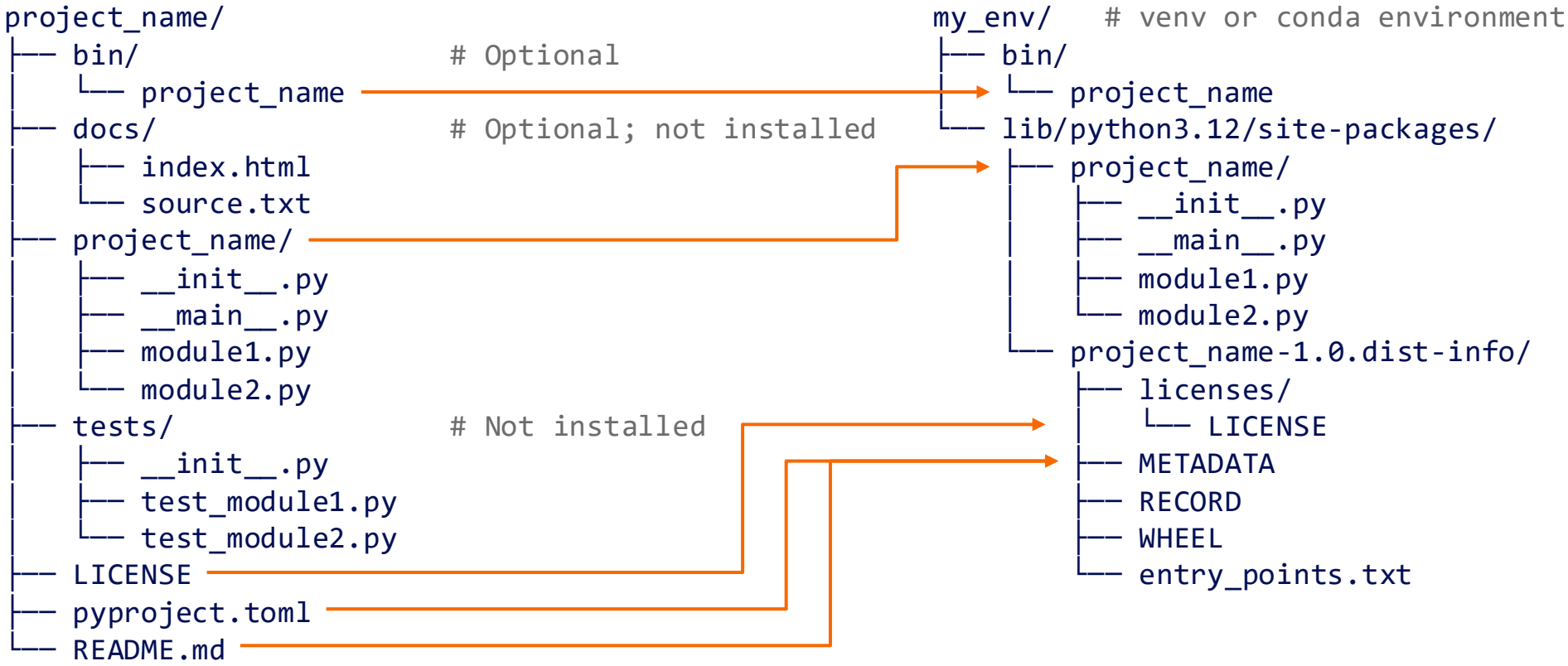
Common source code organization – “flat” layout

```
project_name/
├── bin/                    # Optional
│   └── project_name
├── docs/                   # Optional
│   ├── index.html
│   └── source.txt
├── project_name/
│   ├── __init__.py
│   ├── __main__.py
│   ├── module1.py
│   └── module2.py
├── tests/
│   ├── __init__.py
│   ├── test_module1.py
│   └── test_module2.py
├── LICENSE
├── pyproject.toml
└── README.md
```

README.md	Overview, install steps, usage
LICENSE	Legal terms for using and sharing
project_name/	The package: your Python source code
tests/	Unit tests for the package's modules
bin/	Scripts that launch the project (optional)
docs/	User and developer docs (optional)
pyproject.toml	Metadata, dependencies, build settings

Source: Real Python, “project layout,” Python Best Practices reference, realpython.com/ref/best-practices/project-layout (accessed Oct 2026)

From source tree to installed package



Same layout for venv and conda on Linux/macOS; Windows uses Scripts\ and Lib\site-packages\. Sources: Real Python; PyPA

The src/ layout

```
project_name/
├── bin/                    # Optional
│   └── project_name
├── docs/                   # Optional
│   ├── index.html
│   └── source.txt
├── src/
│   └── project_name/
│       ├── __init__.py
│       ├── __main__.py
│       ├── module1.py
│       └── module2.py
├── tests/
│   ├── __init__.py
│   ├── test_module1.py
│   └── test_module2.py
├── LICENSE
├── README.md
└── pyproject.toml
```

- Alternative to flat layout
- Makes it more clear what packages will be installed
- Probably overkill for most projects (usually you'll only have one top-level package, so src only contains one folder)

Source: Real Python, "project layout," realpython.com/ref/best-practices/project-layout (accessed Oct 2026)

The DON’T DO THIS! layout

```
project_name/
├── doc_index.html
├── doc_source.txt
├── LICENSE
├── main.py
├── module1.py
├── module2.py
├── pyproject.toml
├── README.md
├── test_module1.py
├── test_module2.py
└── tests_init.py
```

- Might be tempting to just stick everything in one folder.
- This is bad!
- Quickly gets unmanageable as project grows
  - E.g., what if you want to have sub-modules? Where do you put multiple doc pages?

Source: Real Python, “project layout,” realpython.com/ref/best-practices/project-layout (accessed Oct 2026)

Describing a project with pyproject.toml

- One config file at the project root, written in TOML
- `[build-system]`: the build backend that builds your package
- `[project]`: name, version, Python version and dependencies
- `[tool.*]`: settings for tools like black or mypy
- pip installs the listed dependencies along with your package

Previously

`setup.py` held this metadata as Python code, and `requirements.txt` listed packages to install. Both still work; new projects use pyproject.toml.

```toml
# pyproject.toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "project_name"
version = "1.0.0"
requires-python = ">=3.10"
dependencies = [
    "numpy>=1.24",
    "requests",
]

[project.optional-dependencies]
test = ["pytest"]
```

For details see: https://packaging.python.org/en/latest/guides/writing-pyproject-toml/

Documenting code with Sphinx

- Sphinx is a tool that makes it easier to create documentation for Python & C code
  - https://www.sphinx-doc.org/en/master/
- It turns plain-text files into searchable documentation for a project
- Uses reStructuredText or MyST Markdown
- Easy to create tables, highlighted code, cross links to other parts of the code base (and even other projects' docs), and to embed and render math equations (using LaTeX)
- Has built-in and third-party themes; extensions like autodoc pull in your code's docstrings
- `pip install sphinx`, then `sphinx-quickstart docs` sets up the docs/ folder
- Example: https://pycbc.org/pycbc/latest/html/inference.html

Deployment via pip

Three steps take a project from your folder to PyPI, where anyone can pip install it.

STEP 1

Describe

pyproject.toml names a build backend (e.g., setuptools) and the metadata: name, version, dependencies.

```
pyproject.toml
```

STEP 2

Build

Creates two archives in dist/: a source distribution (.tar.gz) and a wheel (.whl).

```
python3 -m build
```

STEP 3

Publish

Twine uploads the archives using an API token. Try TestPyPI before the real index.

```
python3 -m twine upload dist/*
```

For details see: https://packaging.python.org/en/latest/tutorials/packaging-projects/

R

R Packaging and Deployment



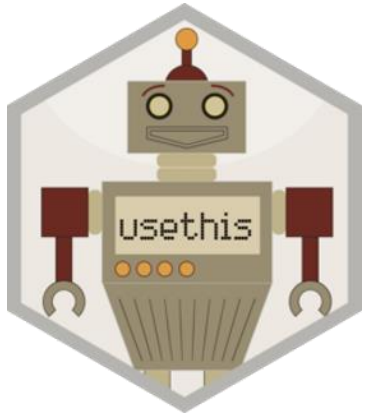
Tools for development and deployment
https://devtools.r-lib.org/



Testing framework for R
https://testthat.r-lib.org/

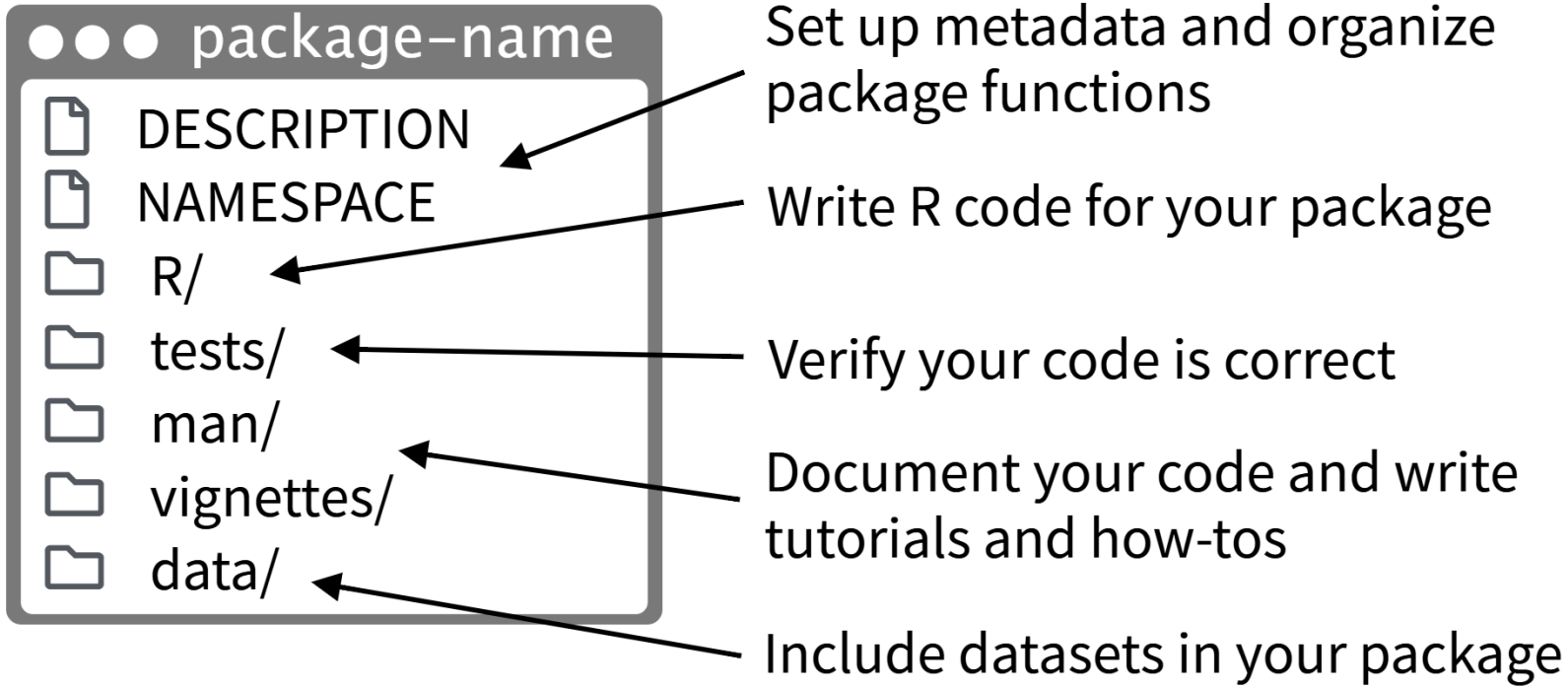


Tools for building a documentation website
https://pkgdown.r-lib.org/



Package setup and helpers for common tasks
https://usethis.r-lib.org/

R source code organization






```
usethis::create_package()
usethis::use_test()
usethis::use_data()
usethis::use_gpl_license()
```

https://rstudio.github.io/cheatsheets/package-development.pdf
https://rstudio.github.io/cheatsheets/html/package-development.html

R source code organization

Documentation

Allows users to call it

Example usage

The actual code

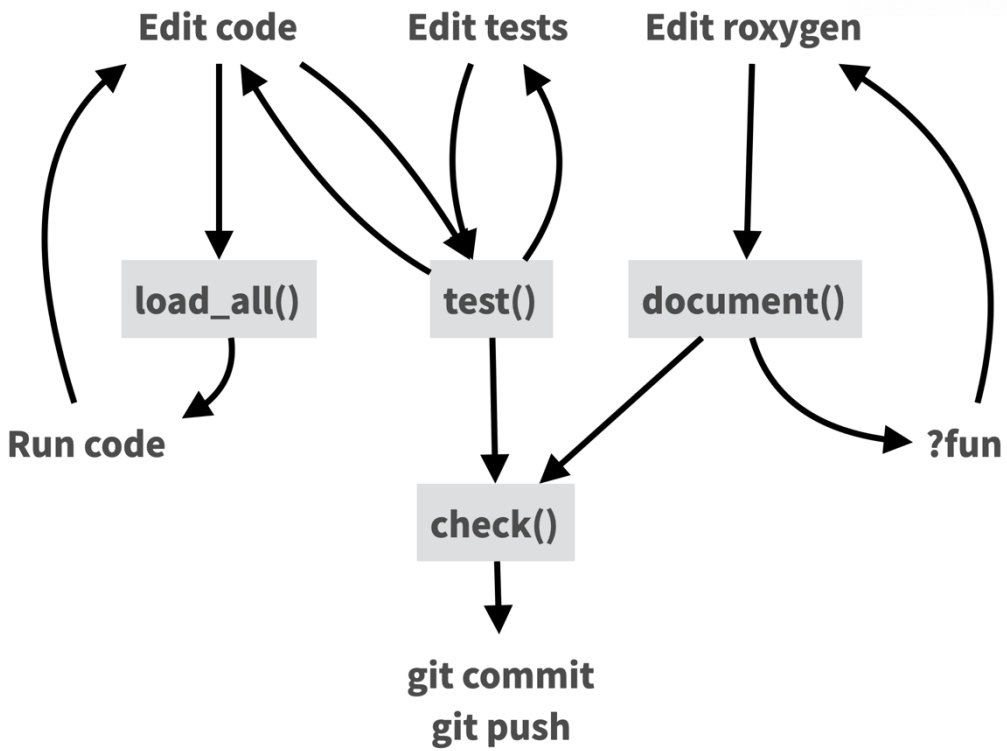
```
#' Add together two numbers
#'
#' @param x A number.
#' @param y A number.
#' @returns The sum of `x` and `y`
#' @export
#' @examples
#' add(1, 1)
add <- function(x, y) {
  x + y
}
```



Convert inline docs to separate markdown files
https://roxygen2.r-lib.org/

https://rstudio.github.io/cheatsheets/package-development.pdf
https://rstudio.github.io/cheatsheets/html/package-development.html

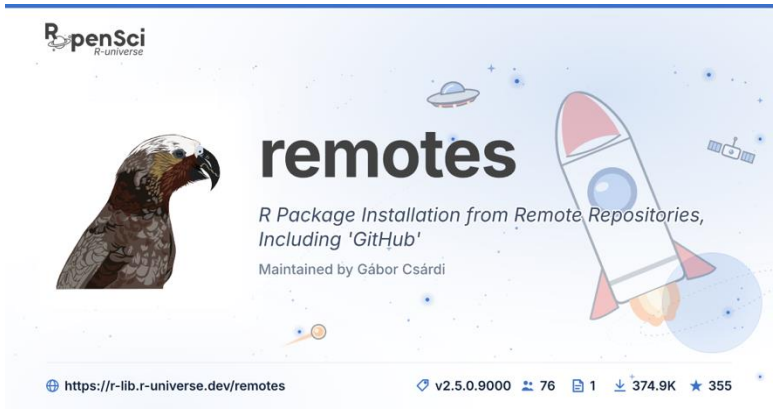
R development cycle

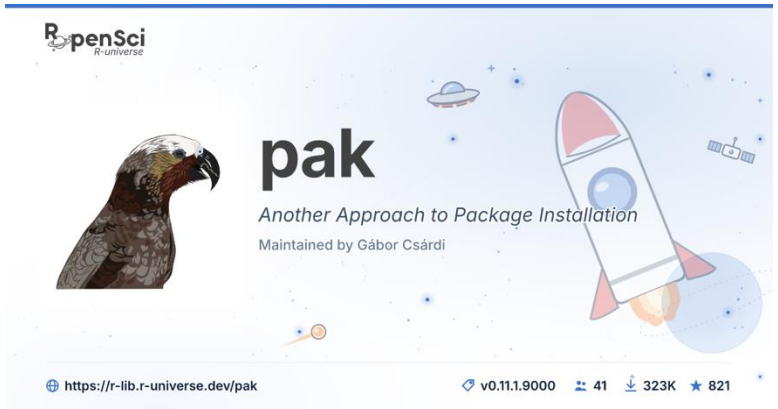




Deployment Options – Source Code (low effort)

- There are now lots of options for installing R packages from source
- Many authors now choose to just host their packages on GitHub and instruct users to install from there

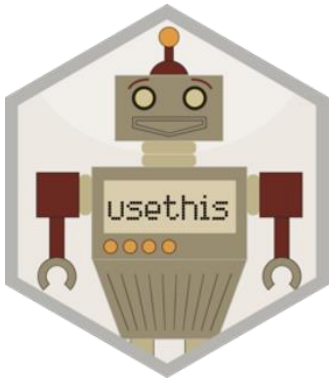




Deployment Options – Binaries (higher effort)

- CRAN: Comprehensive R Archive Network (higher deployment standards, lightweight)
  - Runs checks before accepting packages and regularly for already accepted packages
    - Failing checks results in removal from CRAN (e.g., https://cran.r-project.org/web/checks/check_results_deeptime.html)
  - `usethis::use_release_issue()` makes a handy submission checklist for you (e.g., https://github.com/willgearty/deeptime/issues/109)





Deployment Options – Binaries cont. (higher effort)

- Bioconductor (~exclusively for bio packages)
  - Set up to have periodic “production releases” to ensure all packages are cross-compatible
- R-universe (designed for better discovery and easier deployment)
  - Can continuously build from GitHub with minimal setup (https://willgearty.r-universe.dev/deeptime)
- conda-forge (requires configuration; but lots of software/hardware support)







Websites

Website development – Full Stack

- Full stack websites consist of frontend components (JavaScript, CSS, HTML) and backend components (Python, PHP, Node.js)
- Because of this, they can be simultaneously highly complex and still maintain simple user interfaces
- Full stack websites have databases to store information about users and user interactions, enabling long-term engagement







Common source code organization – Full Stack




```
my_project/
├── manage.py
├── my_project/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── my_app/
│   ├── migrations/
│   │   └── __init__.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── static/
├── media/
└── templates/
```

manage.py: run management commands like runserver

asgi/wsgi: think of this as __main__
settings: defines environmental variables

one or more modules (each one usually has a URL prefix)

migrations: build database

admin: defines structure of administration interface

models: define database structure

urls: define URL-to-view mapping
views: backend logic for web pages
static: CSS/JS
media: downloadable files

templates: HTML

https://medium.com/django-unleashed/django-project-structure-a-comprehensive-guide-4b2ddbf2b6b8

Deployment – Full Stack

- Full-stack websites (e.g., Django, Flask) must be hosted on an active server to run their backend code
- This usually involves several different pieces of software infrastructure to support the database, serving pages to users, receiving requests from users, sending email, etc.
- Tools like Streamlit can be good for prototyping Python-based websites, but are ultimately still hosted on a server
- AI can be a valuable tool for migrating from a Streamlit prototype to a Django/Flask production version

Website development – Static





- Static websites consist of frontend components (JavaScript, CSS, HTML) and some metadata (usually YAML)
- Most importantly, static websites lack databases and therefore lack persistent state (the website usually can not have long-term reactions to how a user interacts with it)
- Because of this, they are usually limited to simple interactions (e.g., blogs, personal websites), but their interfaces can still be quite complex with JavaScript and CSS

Common source code organization – Static



Can use folders to define the URL structure

```
.
├── _config.yml
├── _data
│   └── members.yml
├── _drafts
│   ├── begin-with-the-crazy-ideas.md
│   └── on-simplicity-in-technology.md
├── _includes
│   ├── footer.html
│   └── header.html
├── _layouts
│   ├── default.html
│   └── post.html
├── _posts
│   ├── 2007-10-29-why-every-programmer-should-play-nethack.md
│   └── 2009-04-26-barcamp-boston-4-roundup.md
├── _sass
│   ├── _base.scss
│   └── _layout.scss
├── _site
├── .jekyll-cache
├── .jekyll-metadata
└── index.html # can also be an 'index.md' with valid front matter
```

_config.yml: metadata for the site

_data: various other static data for the site

most content is written in markdown, then converted to HTML

_includes: snippets that can be used in multiple templates

_layouts: structured HTML templates

_sass: basically CSS but behind a compiler

index: the landing page

Taken from: https://jekyllrb.com/docs/structure/
For Hugo: https://gohugo.io/getting-started/directory-structure/

Deployment – Static

- Most static website generators have their own deployment instructions
  1. Generate static content with the engine
  2. Host the static files somewhere
- GitHub Pages is a very popular (and free) strategy for hosting
  - Can set up “Actions” to build the pages automatically on a schedule



Publishing *about* scientific software

Publishing about your software

- There's a growing trend to cite software in publications
- The easiest way to cite a piece of software is with a DOI
- Zenodo is the easiest way to get a DOI for your software
- You can also publish your software in an academic journal
  - Either a general software journal or a field-specific software journal

JOSS
The Journal of Open Source Software



Journal of Statistical Software
September 2016, Volume 73, Issue 2. doi: 10.18637/jss.v073.i02

BRITISH ECOLOGICAL SOCIETY
Methods in Ecology and Evolution

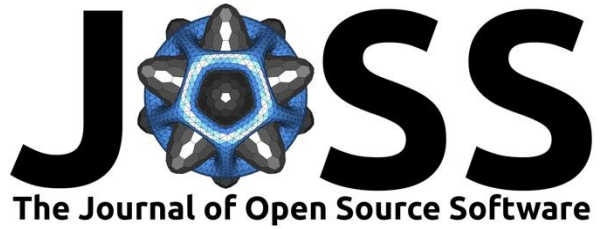
JOSS: The Journal of Open Source Software

https://joss.theoj.org/

- For mature (6+ months) and useful (with evidence) software
- Reviews take place in public GitHub issues
- Submissions consist of:
  1. an open-source repo (e.g., GitHub)
     - OSI-approved license
     - Open development history
  2. a short paper
     - 750-1750 words
     - Required sections (see next slide)

Writing a JOSS Paper

- https://joss.readthedocs.io/en/latest/paper.html



Paper sections

- Summary: description for a non-specialist audience
- Statement of need: what problem does the software solve?
- State of the field: comparison to related existing software
- Software design: explanation of architecture and design choices and why they matter for your application
- Research impact statement: evidence of realized impact
  - For this course, you can be aspirational (what is the potential impact?)
- AI usage disclosure: transparent description of how genAI was used in the software creation, documentation, or paper authoring

Should also include references to related software, important dependencies, relevant research (e.g., for algorithms), etc.

Example paper: https://joss.theoj.org/papers/10.21105/joss.10806